



Bases de datos I

Wilmer Hernán Ortega Erazo

Facultad de ingeniería

Objetivos de Aprendizaje

- ✓ Desarrollar consultas utilizando operadores de agregación SQL.
- ✓ Modificar una base de datos mediante operadores DML del SQL.
- ✓ Resolver una consulta mediante el empleo de vistas en SQL.
- ✓ Definir un campo de una tabla como llave primaria, en SQL.
- ✓ Definir un atributo como llave foránea, en SQL.
- ✓ Describir el concepto de Integridad referencial.

Funciones de Agregación

Existen consultas donde se necesita realizar operaciones que incluyen relaciones en las cuales se agregan valores en una columna. Por **agregación** se entiende una operación que forma **un solo valor a partir de una lista de valores** que tiene la columna.

SQL permite **agregar valores en columnas**, y también agrupar tuplas de una relación conforme a algún criterio, el valor de alguna otra columna, y **agregar luego dentro de cada grupo**.

SQL ofrece **cinco** operadores de agregación:

1. **SUM**, la suma de los valores de una columna.
2. **AVG**, el promedio de los valores de una columna.
3. **MIN**, el valor mínimo de la columna.
4. **MAX**, el valor máximo de la columna.
5. **COUNT**, el número de valores (con duplicados) de la columna.

Funciones de Agregación

CONSULTA: ¿Cuál es el mayor y el menor salario pagado?

```
SELECT MAX(salario), MIN(salario)
FROM Empleado;
```

	max numeric	min numeric
1	1870000.000	1410000.000

CONSULTA: ¿Cuál es el promedio del número de días que los empleados están asignados a la obra 435?

```
SELECT AVG(nroDias)
FROM Asignacion
WHERE idObra = 435;
```

	avg numeric
1	32.33333333333333

Funciones de Agregación

CONSULTA: ¿Cuál es el número total de días utilizados por los plomeros en la obra 312?

```
SELECT SUM(nroDias)
FROM Asignacion, Empleado
WHERE Asignacion.idObra = 312 and
      especialidad = "Plomero" and
      Empleado.codigo = Asignacion.codigo;
```

	sum bigint
1	27

CONSULTA: ¿Cuántas especialidades diferentes hay?

```
SELECT COUNT(DISTINCT especialidad)
FROM Empleado;
```

	count bigint
1	6

Ya que la misma especialidad esta repetida en diferentes filas, es necesario usar la palabra clave “***DISTINCT***” en el query, tal que el sistema **no cuente** la misma especialidad **más de una vez**.

Agrupamiento

Un requerimiento práctico simple es tener los resultados de una consulta **en cierto orden**: alfabético por nombre, descendente por fecha, etc. Esta función la proporciona la cláusula **order by**.

Menos obvia es la necesidad de **agrupar conjuntos de tuplas** que tengan cierto valor en común y aplicar luego **una función de agregación** al conjunto, tal como **COUNT**, **SUM**, etc. en forma separada a cada conjunto.

Para ello se utilizará la cláusula **group by**, colocada después de la cláusula **SELECT**, acompañada de una lista de atributos de **agrupamiento**. Los operadores de agregación que se usen en la cláusula **SELECT** se aplican **únicamente** dentro de los grupos.

Igual que la cláusula **WHERE** aplica en un bloque **SELECT-FROM** como filtro, la cláusula **having** cumple ese papel con los grupos.

Agrupamiento

A continuación, se presenta la sintaxis de la cláusula **GROUP BY** :

```
SELECT
    columna_1,
    columna_2,
    función_agregada(columna_3)
FROM  tablaBase
GROUP BY
    columna_1,
    columna_2;
```

La cláusula **GROUP BY** debe aparecer después de la cláusula **FROM** o **WHERE**. Después del **GROUP BY** hay una columna o lista de columnas separadas. Además de la columna de la tabla, se puede usar una expresión en la cláusula **GROUP BY**.

Agrupamiento

CONSULTA: ¿Para cada supervisor, cual es el mayor salario pagado a un trabajador que le reporta a este supervisor?

```
SELECT codSuperv, MAX(salario)
FROM Empleado
group by codSuperv;
```

codsuperv integer	max numeric
1311	1625000.000
3231	1870000.000
1520	1687500.000
2920	1500000.000

En la consulta, el sistema procede así: primero divide las filas de **Empleado** en grupos, usando la regla que cada fila sea colocada en el mismo grupo si y solo si, tienen el mismo **codSuperv**. Luego se aplica la cláusula **SELECT** a cada grupo. Ya que cada grupo solo puede tener un valor para **codSuperv**, no hay ambigüedad para el grupo. La cláusula **SELECT** llama el **codSuperv**, lo muestra y calcula el **MAX(salario)** desplegándolo, para cada grupo.

El **SELECT** solo pueden contener nombres de columnas usados en una cláusula **group by**, cuando se usan **funciones agregadas**.

Agrupamiento

group by indica qué filas serán agrupadas por un valor común de una(s) **columna(s)**. Se pueden aplicar condiciones a un grupo formado por la cláusula **group by**, enseguida de la cláusula **having**.

CONSULTA: ¿Para cada supervisor que maneje más de un empleado, cual es el mayor salario pagado a un empleado que reporta a este supervisor?

```
SELECT codSuperv, MAX(salario)
FROM Empleado
group by codSuperv
having count(*) > 1;
```

	codsuperv integer	max numeric
1	3231	1870000.000
2	1520	1687500.000

La **diferencia** entre las cláusulas **WHERE** y **having** es que la cláusula **WHERE** aplica a **las filas** mientras la cláusula **having** aplica a **los grupos**.

Modificaciones a la Base de Datos

SQL provee otras formas de proposición que **no devuelven resultado**, sino que **cambian el estado** de la base de datos:

1. **INSERT**, permite la **adición** de tuplas a la relación
2. **UPDATE**, permite **actualizar** los valores de ciertos componentes de determinadas tuplas
3. **DELETE**, permite **eliminar** algunas tuplas en una relación.

La Inserción

La forma fundamental de esta proposición es:

- ♦ Las palabras claves **INSERT INTO**
- ♦ El nombre de la relación **R**
- ♦ Una lista entre paréntesis de los **atributos** de la relación R
- ♦ La palabra clave **values**
- ♦ Una lista de **valores concretos**, uno por cada atributo de la lista.

INSERT INTO R [(A₁, ..., A_n)] values (v₁, ..., v_n);

Modificaciones a la Base de Datos

Otra forma de sintaxis de la operación de inserción es:

```
INSERT INTO R [(A1, ..., An)] subconsulta;
```

Se quiere agregar una nueva tupla a la relación Obras:

```
INSERT INTO Obra  
values (234, “Santa Lucía”, “Apartamentos”, 13, 4);
```

Puedo insertar una nueva tupla, resultado de una subconsulta:

```
INSERT INTO Obra (idObra)  
SELECT idObra FROM Asignación;
```

El efecto de este **INSERT** será que por cada tupla **t** de la relación **Asignación**, se agregará una tupla a la relación **Obra**, con el mismo **idObra** que **t**, y con valores **nulos** en los demás atributos.

Modificaciones a la Base de Datos

Actualización

Una **actualización** es una operación sobre la base de datos donde se **cambian** algunos componentes de una o más tuplas ya existentes en ella. Forma general:

La palabra clave **UPDATE**

Un nombre de relación **R**

La palabra clave **set**

Una **lista de fórmulas** donde un atributo iguala al valor de una expresión o una constante.

La palabra clave **where**

Una **condición**.

UPDATE R set <lista de fórmulas> [where condición];

Modificaciones a la Base de Datos

CONSULTA: Se quiere actualizar el salario de los Empleados dirigidos por el supervisor 1520, incrementándolo en un 10.5%

UPDATE Empleado **set** salario = 1.105 * salario
WHERE codSuperv = 1520;

Eliminación

La *eliminación* permite eliminar tuplas de una relación. Su forma es:

Las palabras claves **DELETE from**

El nombre de relación **R**

La palabra clave **WHERE**

Una **condición**.

Modificaciones a la Base de Datos

La sintaxis de la eliminación es:

DELETE from *R* [*where condición*];

*La ejecución de esta proposición hace que **todas** las tuplas que cumplan con la condición sean **eliminadas** de la relación *R**

CONSULTA: Se quiere eliminar los Empleados que tengan un salario superior al de su supervisor.

```
DELETE from Empleado
WHERE codigo in
(SELECT E.codigo
FROM Empleado E, Empleado S
WHERE E.codSuperv = S.codigo and
      E.salario > S.salario) ;
```


Definición de Vistas

Las relaciones definidas con la proposición **CREATE TABLE** realmente **existen** en la base de datos, con una organización física. Las tablas son **persistentes**.

Hay otra clase de relaciones de SQL, llamadas **vistas**, que **no existen físicamente** y se definen con una expresión muy similar a una consulta. Las vistas **se pueden consultar** como si existieran físicamente, y **en algunos casos** hasta **es posible modificarlas**.

La forma de definir una vista es:

Las palabras claves **CREATE VIEW**

El **nombre de la vista**

La palabra clave **AS**

Una consulta **Seleccione-De-Donde**

CREATE VIEW <nombreVista> **AS** **Seleccione-De-Donde**;

Definición de Vistas

Las vistas permiten:

- **Ocultar información:** permitiendo el acceso a algunos datos y ocultando el resto de la información que no se incluye en la vista. **El usuario solo puede consultar la vista.**
- **Simplificar la administración de los permisos de usuario:** se otorgan permisos para que solo acceda a los datos a través de vistas, en lugar de conceder permisos para acceder a ciertos campos, **así se protegen los datos de cambios en su estructura.**
- **Mejorar el rendimiento:** se puede evitar tipear instrucciones repetidamente almacenando en una vista el resultado de una consulta compleja que incluya información de varias tablas.
- La vista se puede crear con: **un subconjunto de registros y campos de una tabla; la unión de varias tablas;** un resumen estadístico de una tabla; **un subconjunto de otra vista, combinación de vistas y tablas.**

Definición de Vistas

CONSULTA: Se quiere crear una vista que muestra toda la información acerca de los Empleados excepto su salario.

CREATE VIEW VistaEmpleado **AS**

SELECT codigo, nombre, especialidad, codSuperv

FROM Empleado;

La vista se llama **VistaEmpleado** y contiene los atributos **codigo, nombre, especialidad, codSuperv**.

	codigo integer	nombre character varying(30)	especialidad character varying(15)	codsuperv integer
1	1235	M. Franco	Eléctrico	1311
2	1412	C. Nuñez	Plomero	1520
3	2920	R. García	Techos	2920
4	3231	P. Martínez	Armado	3231
5	1520	H. Rivadeneira	Plomero	1520
6	1311	C. Cortes	Pisos	3231
7	3001	J. Barrientos	Repello	3231

Definición de Vistas

La relación **VistaEmpleado** **no contiene tuplas** en el sentido ordinario.

Si consultamos **VistaEmpleado** obtendremos las tuplas correspondientes a la tabla base **Empleado**.

La correspondencia entre **VistaEmpleado** y **Empleado** es **dinámica**: si insertamos un nuevo empleado esta tupla se materializa inmediatamente en **VistaEmpleado**, y si damos de baja un **Empleado**, este desaparecerá de **VistaEmpleado**.

Definición de Vistas

Una vez definida una vista, a ojos del usuario, **ésta se comportará exactamente como cualquier relación de base** en cuanto a las consultas. La siguiente consulta cuenta el número de empleados:

```
SELECT count (*)  
FROM VistaEmpleado;
```

Una vista puede definirse sobre más de una relación. También es posible escribir consultas donde intervengan vistas y tablas base.

CONSULTA: Se necesita información acerca de los electricistas, las obras en que están asignados y la fecha de inicio del trabajo.

```
CREATE VIEW VistaElect AS  
    SELECT codigo, nombre, idObra, fecInic  
    FROM Empleado, Asignación  
    WHERE especialidad = "Electrico" and  
           Empleado.codigo = Asignación.codigo;
```

Definición de Vistas

CONSULTA: Se necesita saber quiénes son los electricistas asignados a la obra 435 y cuando inician su trabajo allí.

```
CREATE VIEW Elect435 (name, dateBegin) AS  
    SELECT nombre, fecInic  
    FROM VistaElect, Asignación  
    WHERE idObra = 435 and  
           VistaElec.codigo = Asignación.codigo;
```

En algunas ocasiones **preferiríamos asignar nuestros propios nombres a los atributos** de una **vista**, en vez de usar los nombres que provienen de la definición de ella en la consulta.

Se puede especificar los atributos de la vista **listándolos**, entre paréntesis, después del nombre de la vista, en la proposición **CREATE VIEW**.

Modificación de Vistas

En unas **pocas circunstancias** es posible ejecutar una inserción, eliminación o actualización en una vista. Esto parece **absurdo** ya que la vista **no existe** como la **tabla base**. ¿Que significa insertar una tupla en una vista? ¿Como recuerda la base de datos lo que contiene la vista? ¿Donde se almacena la tupla?

En muchas vistas esto no puede hacerse. Pero en vistas lo bastante simples, llamadas **vistas actualizables**, podemos traducir la modificación de la vista a una modificación **equivalente** de una tabla base, y así la modificación se hará sobre la tabla base.

SQL2 permite esto gracias a una definición: se admiten modificaciones sobre **vistas** que se definen seleccionando (por medio de **SELECT**, no de la cláusula **SELECT DISTINCT**) **algunos atributos** provenientes de la relación **R** (que también puede ser una vista actualizable).

Control de Seguridad

SQL provee dos sentencias (**grant** y **revoke**), que ayudan a controlar que tipo de acceso se permite a qué usuario, sobre relaciones, sean base o vistas. Para poder ejecutar cualquier operación SQL, **el usuario debe poseer los permisos necesarios**.

Por ejemplo, para ejecutar la consulta:

```
SELECT *  
FROM Empleado;
```

el usuario debe tener autoridad **SELECT** sobre la relación **Empleado**. Para actualizar, se requieren los permisos **INSERT**, **DELETE**, **UPDATE**; existe también permisos para crear, alterar o eliminar una relación y muchos otros tipos.

Los privilegios se otorgan con el comando **grant** y se restringen con el comando **revoke**

Control de Seguridad

Formato del **revoke**:

revoke privilegio [, privilegio] ... | **ALL** [**privileges**])

ON [usuario] **Objeto**

FROM (**public** | usuario [, usuario]...);

privilegios: **alter, delete, index, insert, select y update**

Objeto: puede ser una tabla, vista, sinónimo, etc.

Ejercicios sobre SQL

CLIENTE (idCliente, nomCliente, ingreAñoClie)

EMBARQUE (nroEmbar, idCliente, pesoEmb, nroCamion, codDestino)

CAMION (nroCamion, nombreConductor, placa, marca, modelo)

CIUDAD (codCiudad, nombreCiu, poblacion)

CONSULTAS

- I. Obtener Una lista de embarques con peso mayor de 30 Kg.
- II. Los nombres de los clientes con ingresos anuales superiores a \$ 8'000.000.
- III. Nombre del conductor del camión No. 54.
- IV. Nombre de ciudades que han recibido embarques con peso mayor a 100 Kg.
- V. Nombre e ingreso anual de clientes que han realizado envíos con peso mayor a 80 Kg.
- VI. Nombres de conductores que han transportado embarques con peso mayor a 100 Kg.
- VII. Ciudades que han recibido embarques de clientes con ingreso anual mayor a \$ 12'000.000.

Ejercicios sobre SQL

CLIENTE (idCliente, nomCliente, ingreAñoClie)

EMBARQUE (nroEmbar, idCliente, pesoEmb, nroCamion, codDestino)

CAMION (nroCamion, nombreConductor, placa, marca, modelo)

CIUDAD (codCiudad, nombreCiu, poblacion)

CONSULTAS

- VIII.** Obtener Clientes con ingresos anuales superiores a \$5'000.000, quienes han enviado embarques con un peso menor a 1 Kg.
- IX.** Clientes cuyos embarques han sido transportados por el conductor Roberto Pérez.
- X.** Conductores que han entregado embarques a clientes con ingresos anuales mayores a \$260'000.000 a ciudades con población mayor a 1 millón de habitantes.
- XI.** Nombres de clientes que han realizado embarques usando cada uno de los conductores.
- XII.** Ciudades que han recibido embarques de cada uno de los clientes.
- XIII.** Conductores que han hecho entregas de embarques en todas las ciudades.

Ejercicios sobre SQL

CLIENTE (idCliente, nomCliente, ingreAñoClie)

EMBARQUE (nroEmbar, idCliente, pesoEmb, nroCamion, codDestino)

CAMION (nroCamion, nombreConductor, placa, marca, modelo)

CIUDAD (codCiudad, nombreCiu, poblacion)

CONSULTAS

- I. Obtener Una lista de embarques con peso mayor de 30 Kg.
- II. Los nombres de los clientes con ingresos anuales superiores a \$ 8'000.000.
- III. Nombre del conductor del camión No. 54.
- IV. Nombre de ciudades que han recibido embarques con peso mayor a 100 Kg.
- V. Nombre e ingreso anual de clientes que han realizado envíos con peso mayor a 80 Kg.
- VI. Nombres de conductores que han transportado embarques con peso mayor a 100 Kg.
- VII. Ciudades que han recibido embarques de clientes con ingreso anual mayor a \$ 12'000.000.